SWIFT ENTRY · ARCHITECTURE & DELIVERY PLAN

Multi-Tenant Implementation Plan

One deployment, one database, one login — with every record tagged to its tenant and every role limited to its own company or the whole building.

Document	Prepared	Revision	Scope
Prototype plan	**4 November 2026**	**v1.0**	**Frontend + backend contract**

CONFIDENTIAL — FOR INTERNAL PLANNING

Contents

01 1. Where we are today

02 2. Target model in this app

03 3. Frontend architecture

04 4. Dummy data for the prototype

05 5. Visit status flow (frontend contract)

06 6. Workstreams and file-level changes

07 7. Phased rollout (adapted to a prototype)

08 8. Test plan (the part that protects the demo)

09 9. Backend contract changes (required for production)

10 10. Decisions needed before Phase 1

11 11. Risks specific to this repo

12 12. Definition of done (prototype)

13 13. Suggested first sprint (concrete backlog)

> **Scope:** this repository is the **frontend only** (React 19 + Vite, no TypeScript, no Redux). The Laravel API and database are **not in this repo**. This plan therefore separates:
>
> 1. what we can build and demo here with dummy data, and
> 2. what the backend must enforce before this is safe in production.
>
> **Golden rule:** the frontend can hide and filter, but it **cannot** be the security boundary. Tenant isolation must ultimately be enforced by the API + database (row-level security or a single scoped query layer).

---

1. Where we are today

Facts taken from the current codebase:

Area	Current state	Impact on tenancy
Repo	Frontend only, ~167 JS/JSX files, ~283 API call sites	All tenant logic must be introduced at the API/context layer, not by rewriting 190 pages
Auth	`AuthContext.jsx` logs in by probing 5 role endpoints in order and injects the role client-side	Conflicts with "one login address; the account decides scope and tenant"
Route guard	`PrivateRoute.jsx` ignores `requiredRole` entirely	Any logged-in user can open any dashboard by URL — must be fixed first
Data layer	One axios instance in `src/context/api.jsx`, hardcoded production URL	Single insertion point for tenant headers and the mock adapter
State	React Context + local state; no Redux	Tenant context fits the existing pattern
Styling	Bootstrap + heavy inline styles; design tokens dormant	New screens must follow existing admin sidebar/topbar components
Tests	No test framework, 118 lint errors baseline	Need at least a cross-tenant test harness before Phase 2 ships
Notifications	`/notifications.{userId}` private channels	Tenant ID must be added to payloads; channel names can stay user-based, but payload must carry `tenant_id`
Existing statuses	`scheduled / pending / completed / cancelled / missed`	Must map to the new status flow without breaking existing tables

1.1 Foundation blockers to fix before adding tenancy

These are already broken and will undermine any tenant work:

1. `PrivateRoute` **does not check roles** (`src/customRoute/PrivateRoute.jsx:5`). Fix with a real guard.
2. **Login probes every role endpoint** (`src/context/AuthContext.jsx:273–374`). Replace with a single login contract (or a transitional adapter) that returns `scope`, `tenant`, `permissions`.
3. `normalizeUser` **spreads** `...rawUser` **last** (`AuthContext.jsx:95–96,123`), so raw fields can overwrite normalized ones — including new tenant fields. Fix the merge order.
4. **No 401 interceptor** (`api.jsx`). A suspended tenant must be logged out and told why.
5. `echo.js` **captures the token once at module load** (`echo.js:20`). Tenant-aware realtime needs a fresh token.
6. **Cached** `localStorage.user` **is trusted before** `/me` **resolves** (`AuthContext.jsx:133`). On refresh, tenant/scope could be stale after a role change.
7. **~20 endpoints are missing a leading slash** (`api.get("admin/staff")`), which resolves outside `/api`. New tenant endpoints must not repeat this.

These are **Phase 1** work; do not build tenant screens on top of them.

---

2. Target model in this app

2.1 Vocabulary

Concept	Meaning in the app
Building	The deployment. One domain, one database, one set of shared services. Holds building-wide settings and branding defaults.
Tenant	A company inside the building. Has its own people, visitors, visits, optional front desk, settings overrides and branding.
Scope	`building` or `tenant`. Stored on the user.
Building user	`scope = building`, `tenant_id = null`. Sees all tenants (optionally filtered).
Tenant user	`scope = tenant`, `tenant_id = <id>`. Sees only that tenant.
Active tenant filter	A building admin's *view filter*, **not** an identity change. Must never be settable by a tenant user.

2.2 Role and scope matrix

Role	Scope	Tenant	Reaches	Notes
`admin`	`building`	`null`	All tenants	Creates/suspends tenants, building settings, gate policy, all visits
`admin`	`tenant`	required	Own tenant	Staff, tenant roles, tenant settings, own visits
`front_desk`	`building`	`null`	All tenants	Shared reception; verifies codes, check-in/out
`front_desk`	`tenant`	required	Own tenant	Optional per tenant (e.g. floor reception)
`security`	`building`	`null`	All tenants	QR + visit code verification only
`self_service`	`building`	`null`	Whole building	Kiosk/portal; visitor never chooses a tenant
`staff`	`tenant`	required	Own tenant	Registers visitors; is the host

Forbidden combinations to enforce in UI **and** API:

- tenant user with `tenant_id = null`
- `security` or `self_service` with tenant scope
- tenant admin creating a building-scoped user
- tenant user changing the active tenant filter

2.3 Recommended route structure

Reuse the existing admin shell rather than cloning ~190 files.

Route	Who	Purpose
`/admin-dashboard`	building admin	Building overview, all tenants

Route	Who	Purpose
`/admin-dashboard/tenants`	building admin	Tenant list, create, suspend
`/admin-dashboard/tenants/:tenantId`	building admin	Tenant detail: people, visits, settings, usage
`/admin-dashboard/building-settings`	building admin	Gate policy, features, retention, branding defaults
`/admin-dashboard/...` (existing)	building admin	Existing visitors, staff, reports, settings
`/tenant-dashboard`	tenant admin	Tenant overview, own visits
`/tenant-dashboard/staff`	tenant admin	Own staff + CSV bulk upload
`/tenant-dashboard/visits`	tenant admin	Own visits only
`/tenant-dashboard/settings`	tenant admin	Allowed overrides, branding, own front desk
`/tenant-dashboard/front-desk`	tenant front desk	Own tenant's reception (only if enabled)
existing role dashboards	shared users	Unchanged routes, tenant-aware data

Decision: one admin shell (`AdminLayout` + `AdminSidebar`), with nav items filtered by `scope` . The building admin sees a "Tenants" section; the tenant admin sees a reduced menu and a persistent tenant badge. This avoids duplicating the admin UI.

Tenant switcher: only for building scope. It sets `viewingTenantId` in `TenantContext` and appends `?tenant_id=` to list/analytics requests. It must reset on logout and be ignored/forbidden for tenant users.

---

3. Frontend architecture

3.1 New context: `TenantContext`

`src/context/TenantContext.jsx`

```
{
  building: { id, name, logo, timezone, gatePolicy, features },
  tenants: [],                 // only populated for building scope
  activeTenant: null,          // identity tenant for tenant users
  viewingTenantId: null,       // building-admin filter only
  setViewingTenantId(id),
  can(permission),             // scope-aware helper
  isLoading, error
}
```

Rules:

- `activeTenant` comes from `/me` / login response, never from a URL parameter.
- `viewingTenantId` is persisted in memory (optionally `sessionStorage`), cleared on logout.
- A tenant user's `viewingTenantId` is hard-pinned to their own tenant; the setter is a no-op for them.

3.2 Auth contract (target)

Single login:

```
POST /api/auth/login
{ "email": "...", "password": "...", "recaptcha_token": "..." }
```

Response:

```
{
  "token": "...",
  "user": {
    "id": 42,
    "name": "Amara Okafor",
    "email": "amara@acme.example",
    "role": "admin",
    "scope": "tenant",
    "tenant_id": 2,
    "tenant": { "id": 2, "name": "Acme Health", "status": "active" },
    "building_id": 1,
    "permissions": ["tenant.staff.manage", "tenant.settings.manage"]
  }
}
```

`GET /api/me` returns the same user shape and is the source of truth on refresh. Until the backend ships this, keep a **transitional adapter** in `AuthContext`:

- try single login first (feature flag),
- fall back to the existing 5-probe loop only in legacy mode,
- derive a default `scope` / `tenant_id` for legacy accounts (building admin → building; everyone else → the deployment's default tenant).

3.3 Scoped API client

Extend `src/context/api.jsx`:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;

  // Building-level filter only. Server must ignore/reject for tenant users.
  if (config.tenantScoped) {
    const viewing = sessionStorage.getItem("viewingTenantId");
    if (viewing) config.params = { ...config.params, tenant_id: viewing };
  }
  return config;
});
```

Do not send `X-Tenant-Id` as an identity claim. Identity comes from the token. `tenant_id` is a filter for building admins only.

Add:

- response interceptor: `401` → logout with "session expired"; `403 TENANT_SUSPENDED` → logout with reason; `403 TENANT_FORBIDDEN` → toast + safe redirect.
- a small `withTenantScope(params)` helper so list screens don't hand-roll filters.

3.4 Guard

Rewrite `PrivateRoute`:

```
<PrivateRoute requiredRole="admin" requiredScope="building">
```

Checks, in order:

1. authenticated? → `/`
2. `user.scope` matches `requiredScope` (if provided)?
3. `user.role` matches `requiredRole` (if provided)?

4. tenant user's `tenant_id` present and active?
5. tenant user trying to open another tenant's detail → 403 screen, not a redirect loop.

Add a `<TenantBoundary tenantId>` wrapper for detail pages that asserts `tenantId === user.tenant_id` for tenant scope.

3.5 Central UI primitives

Create once, reuse everywhere:

Component	Purpose
`<TenantBadge tenant={...} />`	Colour-coded tenant chip on every visit/user row
`<TenantSwitcher />`	Building admin only; "All tenants" + tenant list
`<ScopeBanner />`	"Viewing: Acme Health — tenant data only" for tenant admins
`<TenantGuard>`	Inline permission/tenant check for actions
`<EmptyForScope />`	Empty state that never leaks counts from other tenants

3.6 Settings inheritance

Resolution order:

```
effective = tenantOverride ?? buildingDefault
```

Expose `useEffectiveSetting(key)` from `TenantContext`. UI shows:

- **Inherited** (read-only, "Using building default")
- **Overridden** (editable, "Reset to building default") — only when the building allows the override.

Building admin controls which keys are overridable (`allow_tenant_override`).

---

4. Dummy data for the prototype

Because there is no backend in this repo, the prototype needs a **mock data layer** that can be switched on without touching page code.

4.1 Approach

Add:

```
src/mocks/
  db.js           // in-memory store, reset()
  seed.js         // deterministic fixtures (no faker dependency)
  adapter.js      // axios adapter / router: method + path -> handler
  latency.js      // small artificial delay, occasional 403 scenarios
  scenarios.js    // named fixtures: normal, cross-tenant, suspended-tenant
  README.md
```

Enable with:

```
VITE_USE_MOCK_API=true
VITE_MOCK_SCENARIO=normal
```

In `api.jsx`:

```
if (import.meta.env.VITE_USE_MOCK_API === "true") {
  api.defaults.adapter = mockAdapter;   // no page changes required
}
```

Properties:

- **No new runtime dependency** (we implement a tiny router; no MSW required).
- **Deterministic**: fixed IDs, fixed dates relative to "today" at seed time.
- **Resettable**: `resetMockDb()` restores the seed; exposed in a dev-only panel.
- **Persona switcher**: dev-only floating panel to log in as any seeded user, reset data, or switch scenario.
- **Never bundled for production**: guard with `import.meta.env.DEV` and the env flag; add a build-time warning/assert.

4.2 Seed specification

Building

- `id: 1`, name "Swift Entry Tower", 12 floors, timezone `Africa/Lagos`
- Gate policy: shared front desk **on**, security **on**, portal **on**, QR verification required
- Branding: navy `#053561`, logo `/pwa-512x512.svg`
- Retention: 90 days

Tenants (3 active + 1 suspended for testing)

ID	Name	Floor	Status	Own front desk	Branding override
1	Acme Health	3–4	active	yes	teal
2	Northwind Legal	5	active	no	inherit
3	Globex Labs	6	active	no	purple
4	Initech (suspended)	7	suspended	no	inherit

Users (password `Demo1234!` for all; shown in the dev panel only)

Role	Scope	Tenant	Email
admin	building	—	`building.admin@swiftentry.test`
front_desk	building	—	`reception@swiftentry.test`
security	building	—	`security@swiftentry.test`
self_service	building	—	`kiosk@swiftentry.test`
admin	tenant	Acme	`admin@acme.test`
admin	tenant	Northwind	`admin@northwind.test`
admin	tenant	Globex	`admin@globex.test`
front_desk	tenant	Acme	`reception@acme.test`
staff	tenant	Acme	`staff1@acme.test` … `staff4@acme.test`
staff	tenant	Northwind	`staff1@northwind.test` …
staff	tenant	Globex	`staff1@globex.test` …

Visitors — cross-tenant trap (must be in the seed from day one)

- `Ada Lovelace` has a completed visit to **Acme** last week and a scheduled visit to **Globex** tomorrow.
- `Grace Hopper` has a cancelled visit to **Northwind** and a checked-in visit to **Acme** today.
- Same email/phone on both records, **separate visitor rows per tenant** (or one identity row + per-tenant visitor profile — see §10, decision D3).

Any screen that shows Ada's Globex visit to an Acme admin is a **P0 bug**.

Visits (~60 rows)

Spread across:

- statuses: `registered`, `verified`, `checked_in`, `checked_out`, `cancelled`, `missed`, `expired`
- sources: `pre_registered`, `walk_in`
- dates: today, this week, this month, last month
- hosts: at least 3 staff per tenant
- gates: shared front desk, security, portal, tenant front desk
- 2 group visits, 1 VIP, 1 blacklisted visitor (per tenant)

Gate/audit events

One row per status transition: `who`, `role`, `scope`, `tenant_id`, `gate` (`front_desk_shared`, `front_desk_tenant`, `security`, `self_service`), `timestamp`, `from`, `to`.

Notifications

Per user, tenant-tagged. Seed at least one notification that would be wrong to show across tenants (e.g. "Ada arrived for Globex") to prove the filter.

Settings

- Building defaults: purposes, floors, visit rules, notification templates.
- Acme overrides: branding teal, own front desk on, visitor email copy.
- Northwind: inherits everything.
- Globex: branding purple, everything else inherits.

4.3 Demo personas

The dev panel should offer one-click login as:

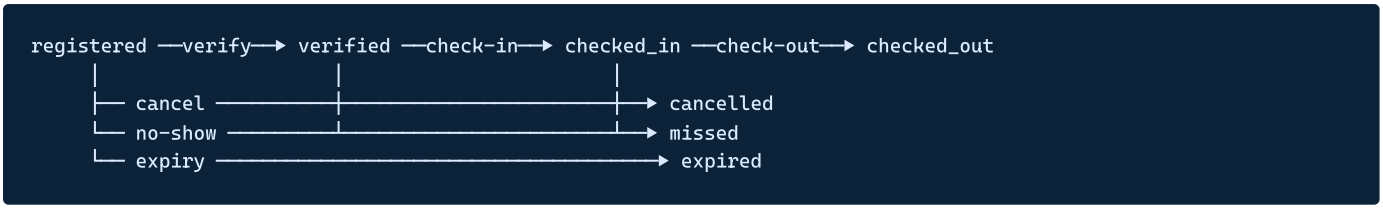
1. Building admin
2. Acme tenant admin
3. Northwind tenant admin
4. Shared front desk
5. Security
6. Acme staff
7. Globex staff
8. Visitor kiosk

This makes the isolation story demonstrable in under a minute.

---

5. Visit status flow (frontend contract)

The existing app uses `scheduled / pending / completed / cancelled / missed`. The new flow removes tenant approval. Proposed canonical statuses:



- **Pre-registered** starts at `registered` (staff already approved).
- **Walk-in** starts at `registered` and is immediately `verified` + `checked_in` by the portal (single atomic action), or by the front desk assisting.
- **No approve/decline step** anywhere.
- Every transition writes an audit event with `tenant_id`, actor, gate and timestamp.

Legacy mapping (for existing tables and API responses):

Legacy	Canonical
`scheduled`	`registered`
`pending`	`registered` (approval retired; label "Awaiting verification")
`completed`	`checked_out`
`cancelled`	`cancelled`
`missed`	`missed`

Keep the legacy field for compatibility and add `status_v2`; the UI reads a single adapter function `toCanonicalStatus()`.

QR/visit code: the frontend never generates the security token. It displays the token/QR returned by the API. `GenerateQRCode.jsx` currently builds `guest-${email}` client-side — replace with the server-issued signed token. The visit code must be unique **across the building**.

---

6. Workstreams and file-level changes

Workstream A — Foundation (Phase 1)

Change	Files
Tenant context + hooks	**new** `src/context/TenantContext.jsx`, `src/hooks/useTenantScope.js`
Auth contract + normalization	`src/context/AuthContext.jsx`
Real route guard	`src/customRoute/PrivateRoute.jsx`, `src/App.jsx`
Tenant-aware API client	`src/context/api.jsx`
401/403 handling	`src/context/api.jsx`, `src/context/AuthContext.jsx`
Realtime token + tenant payload	`src/context/echo.js`, `src/context/NotificationContext.jsx`
Mock layer	**new** `src/mocks/*`, `.env.example`
Dev persona/reset panel	**new** `src/components/dev/DevTenantPanel.jsx` (DEV only)

Workstream B — Read-path isolation (Phase 2)

Change	Files
Tenant filter on every list/analytics call	all visit tables, `AdminDashboard.jsx`, `AnalyticalAndReport.jsx`, history pages
Tenant badge on rows and detail headers	`src/components/TenantBadge.jsx` + table components
Tenant-scoped notifications	`NotificationContext.jsx`, notification pages
Tenant-scoped exports	`AnalyticalAndReport.jsx`, data export settings
Central scoped query helper	**new** `src/utils/tenantQuery.js`
Cross-tenant test harness	**new** `src/mocks/__tests__/isolation.*` or dev inspector

Workstream C — Building admin (Phase 3)

Screen	New files
Tenant list (status, floor, people, visits, usage)	`src/pages/admin/tenants/TenantList.jsx`
Create/edit tenant	`src/pages/admin/tenants/TenantForm.jsx`
Tenant detail	`src/pages/admin/tenants/TenantDetail.jsx`
Suspend/reactivate + login block	`TenantDetail.jsx` + confirmation modal
Building settings (gate policy, features, retention, overrides)	`src/pages/admin/settings/BuildingSettings.jsx`
Tenant switcher	`src/components/TenantSwitcher.jsx`, `AdminSidebar.jsx`, `AdminTopHeader.jsx`
Nav filtering by scope	`src/pages/admin/include/AdminSidebar.jsx`

Workstream D — Tenant admin (Phase 4)

Screen	New files / changes
Tenant dashboard (own visits only)	`src/pages/tenant/TenantDashboard.jsx`
Staff list + add/edit/deactivate	`src/pages/tenant/TenantStaff.jsx`
CSV bulk upload (tenant roles only, duplicate check inside tenant)	`src/pages/tenant/TenantStaffBulkUpload.jsx`, reuse `BulkUploadModal.jsx`
Tenant settings + override/reset UI	`src/pages/tenant/TenantSettings.jsx`
Tenant branding preview	`src/pages/tenant/TenantBranding.jsx`
Optional tenant front desk toggle	`TenantSettings.jsx`
Tenant visits / history	`src/pages/tenant/TenantVisits.jsx`

Workstream E — Gate flow + audit (Phase 5)

Change	Files
Enforce "front desk verifies code; security verifies QR + code" as a building setting	`ConfirmVisitor.jsx`, `SecurityDashboard.jsx`, `GenerateByUniqueCode.jsx`
Remove client-generated QR	`GenerateQRCode.jsx`, registration pages
Walk-in portal: visitor never picks tenant	`VisitorWalkInRegistration.jsx`
Status ladder + audit timeline component	**new** `src/components/VisitTimeline.jsx`

Change	Files
Gate audit log screen	`src/pages/admin/audit/GateAuditLog.jsx`
Host notification on entry	gate confirm flows

---

7. Phased rollout (adapted to a prototype)

Phase 0 — Baseline (1–2 days)

- Branch + snapshot lint (`118 errors / 51 warnings`).
- Add `.env.example` with `VITE_USE_MOCK_API` , `VITE_MOCK_SCENARIO` , `VITE_API_BASE_URL` .
- Define the canonical JSON contracts (login, `/me` , tenant, visit, audit).
- Agree on the 4 open decisions in §10.

Gate 0: contracts reviewed; nobody writes UI before the user shape is agreed.

Phase 1 — Invisible foundations (3–5 days)

- Fix `PrivateRoute` , `normalizeUser` , login adapter, API interceptors.
- Add `TenantContext` + mock store + seed + dev persona panel.
- Everything still looks the same; the app can run in `mock` or `live` mode.

Gate 1 (must pass):

- Two test tenants exist.
- Logging in as Acme admin cannot open a Globex detail URL (guard blocks).
- Switching mock/live is one env variable.
- No increase in lint baseline.

Phase 2 — Read-path isolation (3–4 days)

- Every list, dashboard KPI, report, export and notification is tenant-filtered.
- Tenant badge visible wherever a tenant exists.
- Cross-tenant test harness green.

Gate 2 (must pass): the cross-tenant matrix in §8 passes for **every** list/detail/export screen. One leak stops the phase.

Phase 3 — Building admin (4–6 days)

- Tenant CRUD, suspend/reactivate, tenant detail, building settings, tenant switcher.
- Tenant creation seeds an empty tenant admin invite.

Gate 3: create a new tenant, invite its admin, log in as that admin, see an empty tenant; suspend it, confirm login is blocked with a clear message; existing tenants unaffected.

Phase 4 — Tenant admin (4–6 days)

- Tenant dashboard, staff management, bulk upload, tenant settings, branding, optional front desk.
- Inherit/override/reset behaviour visible.

Gate 4: tenant admin can manage only their own staff and settings; CSV with a building-role row is rejected; duplicate emails are checked inside the tenant only.

Phase 5 — Gate flow, audit and polish (2–3 days)

- Status ladder, QR token from API, walk-in portal, audit timeline, host notification.
- Demo script + seed polish.

Gate 5: both end-to-end scenarios run cleanly in the demo:

1. Staff pre-registers → shared front desk verifies → host notified → tenant dashboard says "arrived in building".
2. Visitor walks in → portal finds host → checks in → host notified → no approval step.

Total estimate: 17–26 developer-days for the prototype, excluding backend work.

---

8. Test plan (the part that protects the demo)

8.1 Cross-tenant matrix

For each pair `(actor tenant, target tenant)` where `actor ≠ target`, assert **403 or empty** — never data:

Screen	Acme actor	Northwind actor	Globex actor
Visit list / search	no Northwind/Globex rows	no Acme/Globex rows	no Acme/Northwind rows
Visit detail by ID	403	403	403
Visitor profile	403	403	403
Dashboard KPIs	counts only own	only own	only own
Reports / export	own rows only	own rows only	own rows only
Notifications	own only	own only	own only
Staff list	own only	own only	own only
Settings override	own only	own only	own only
Bulk upload	own only	own only	own only
Audit log	own only	own only	own only

Also test the **shared** roles: building front desk, security and portal must see all tenants; tenant front desk must see only its own.

8.2 Role matrix

- Every protected route: correct role + correct scope.
- Wrong role → 403 page (not a redirect loop).
- Wrong scope (tenant user opening building-only screen) → 403.
- Missing `tenant_id` on a tenant user → forced logout, never "see all".

8.3 Status and audit

- Every transition is allowed only from valid predecessor states.
- Every transition writes exactly one audit row with actor, gate, tenant, timestamp.
- Cancelled/missed/expired visits cannot be checked in.
- Walk-in creates `registered → verified → checked_in` atomically.

8.4 Edge cases

- Suspended tenant: login blocked; existing history retained for building admin.

- Deactivated staff: cannot log in; their past visits remain.
- Same email in two tenants: allowed (email unique within building, not globally).
- Visitor who visits two tenants: separate records; no cross-tenant hint.
- Deep link refresh: `/me` resolves before protected UI renders.
- Logout clears `viewingTenantId`.

8.5 Recommended tooling

Add **Vitest + React Testing Library** (uses the existing Vite config) for the isolation matrix. If adding dependencies is not allowed for the prototype, build a dev-only **"Tenant Isolation Inspector"** page that runs the same assertions against the mock DB and shows pass/fail. Either way, the matrix must be executable, not manual.

---

9. Backend contract changes (required for production)

The frontend plan only works if the API does the following. Share this list with the backend team.

#	Requirement
1	`POST /auth/login` returns `role`, `scope`, `tenant_id`, `building_id`, `permissions`; role is **not** inferred by probing endpoints
2	`GET /me` returns the same shape; token carries `scope` + `tenant_id`
3	Every tenant-owned table has an indexed `tenant_id`
4	Global tenant scope / RLS on all queries, not per-controller forgetfulness
5	Tenant users cannot set `tenant_id`; server derives it from the token. Building users may pass it as a filter
6	Suspended tenant → `403 TENANT_SUSPENDED` on login and every request
7	Email uniqueness enforced per building (tenant users) — clarify global uniqueness for building users
8	Visit codes unique across the building; QR is a signed token `{visit_id, tenant_id, exp}`
9	Settings endpoint returns effective values + source (`building_default` / `tenant_override`)
10	Bulk upload validates roles against the caller's scope
11	Reports/export accept `tenant_id` only for building scope
12	Audit table records actor, gate, tenant, from/to status, timestamp
13	Notification payloads and queued jobs carry `tenant_id`; emails use tenant branding and name the tenant
14	Uploaded files stored under a tenant path with access checks
15	Retention job per building; deletion-on-request endpoint
16	Legacy deployments: one building + one default tenant; old domain redirects to the single login

---

10. Decisions needed before Phase 1

These change the data model and should be settled now:

1. **Admin shell:** one `/admin-dashboard` with scope-driven nav (recommended), or separate `/tenants-dashboard`?
2. **Canonical status names:** use the proposed `registered / verified / checked_in / checked_out` (recommended) or keep legacy `scheduled / pending / completed` and add a `source` flag only?
3. **Visitor identity:** one global visitor row shared across tenants, or one visitor row per tenant (recommended for privacy — a tenant must not see another tenant's visit history)? If global, every read must be join-scoped through visits.
4. **Tenant-level front desk:** does it share the same `front_desk` role with `scope=tenant` (recommended), or a separate `tenant_front_desk` role?
5. **Building admin viewing a tenant:** pure filter (recommended) or an explicit "impersonation" mode with an audit entry?
6. **Mock in builds:** allow `VITE_USE_MOCK_API` in preview/staging, or restrict to `npm run dev` only (recommended)?

---

11. Risks specific to this repo

Risk	Why it matters here	Mitigation
Frontend-only isolation	UI hiding is not security; the live API is remote	Treat mock isolation as a demo; ship backend RLS before real tenants
Huge duplicated page surface	8 near-duplicate registration pages, 12+ visit tables	Central `tenantQuery` + `TenantBadge`; don't patch page by page
Auth probe login	Client-assigned role and tenant can drift	Single-login contract + `/me` source of truth
Cached user	Stale tenant after role change	Always refresh `/me` before rendering protected UI
Realtime channels	`echo.js` token frozen; tenant payload leaks	Fresh token function + tenant-tagged payloads; keep polling fallback
Deep links	Detailed pages fetch by ID	`TenantBoundary` + server-side 403; never rely on list filtering
Suspended tenant leaks	Sessions may stay alive	401/403 interceptor + server check on every request
Dummy data mistaken for real	Demo could be deployed	Hard-fail build if mock flag is on outside dev; visible "DEMO DATA" banner
Existing known bugs	118 lint errors, broken private route, missing slashes	Fix the guard first; keep lint baseline snapshot
Shared gate accounts	Actions need attribution	One login per person; audit actor on every check-in/out

---

12. Definition of done (prototype)

The prototype is demo-ready when:

- [ ] One login screen leads to the right scope and tenant.
- [ ] Building admin can create, suspend and view tenants, and filter every list by tenant.
- [ ] Tenant admin sees only their own staff, visits, settings and reports.
- [ ] Shared front desk, security and portal see all tenants; tenant front desk sees only its own.

- [ ] A pre-registered visitor goes `registered → verified → checked_in → checked_out` with an audit trail.
- [ ] A walk-in checks in on the portal and the host is notified; no approval step.
- [ ] Cross-tenant matrix passes for every screen, including direct URL access and export.
- [ ] Mock mode can be switched off without page changes.
- [ ] Seed reset restores the exact demo state.
- [ ] No increase in lint baseline; production build passes.

---

13. Suggested first sprint (concrete backlog)

1. Snapshot lint; create branch `feat/multitenancy-foundation`.
2. Add `.env.example`; wire `VITE_USE_MOCK_API` into `api.jsx`.
3. Add `src/mocks/db.js`, `seed.js`, `adapter.js`; seed building + 3 tenants + 8 users + 2 trap visitors + 20 visits.
4. Add `TenantContext`, `useTenantScope`, `TenantBadge`, `TenantSwitcher` (building only).
5. Rewrite `PrivateRoute` with role + scope; update `App.jsx` props.
6. Add `scope`, `tenant_id`, `permissions` to `normalizeUser`; add single-login adapter; add `/me` refresh.
7. Add 401/403 interceptors and suspend handling.
8. Build the dev persona panel (8 one-click logins + reset).
9. Write the cross-tenant matrix as an executable check.
10. Demo: log in as Acme admin, try to open a Globex visit URL, show 403; log in as building admin, show the same visit.

That demo is the proof that the foundation is correct before any new tenant UI is built.